Lecture 5

Questions to an oracle

Objectives. *Lecture 2 showed Deutsch's one-query quantum advantage. This lecture asks:* can the query gap grow exponentially? *In Simon's problem, $n+2$ quantum queries decide whether a hidden string s is zero, with bounded error (Theorem 5.6, the main result). We fix the query model and its classical competitor: every classical bounded-error algorithm needs of the order of $2^{n/2}$ queries (Theorem 5.9, cited). The separation is proved, relative to an oracle.*

By the end of this lecture you will be able to:

- Define the query model, and show on Deutsch–Jozsa's problem why a count of queries must name its competitor.
- Show that garbage which tells branches apart destroys their interference, and that constant garbage changes nothing.
- Compute the Hadamard transform by cancellation, and prove that Simon's circuit returns a uniform y with $y \cdot s = 0$, and that $n+2$ runs decide whether $s = 0$, with error probability below $\frac{1}{4}$. An incorrect verdict is possible only when $s = 0$.
- State the classical $\Omega(2^{n/2})$ bound with its idea, and the status of the separation.

5.1 The query model

Definition 2.5 made the oracle U_f of a function and called each use of it a query. To compare algorithms by their queries, both sides of the comparison must be fixed.

Definition 5.1 (The query model)**.** A *problem* in the query model is a set $\mathcal{F}$ of functions $f\colon \{0,1\}^n \to \{0,1\}^m$, the *promise*, and a property of f to be decided.

- A *quantum query algorithm* is a procedure (§ 3.3) in which f enters only through U_f, applied to $n+m$ of its qubits.
- A *classical query algorithm* chooses strings x, each from the answers before it and from random bits, and receives $f(x)$.

The *cost* is the largest number of queries, uses of U_f or strings chosen, over all runs and all $f \in \mathcal{F}$. An algorithm decides the property *exactly* if it answers correctly with probability 1 for every $f \in \mathcal{F}$, and *with bounded error* if it does so with probability at least $\frac{2}{3}$ for every $f \in \mathcal{F}$.

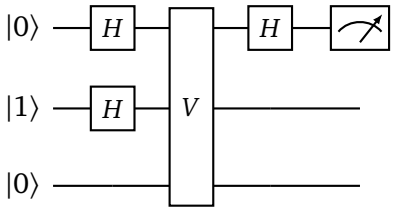
Everything but the queries is free: gates, readings, and any computation between queries. The model asks how much must be learned about f, not how hard it is to compute with what was learned, and in it lower bounds can be proved (§ 5.6).

The competitor must be named. In Deutsch–Jozsa's problem, $f\colon \{0,1\}^n \to \{0,1\}$ is promised to be constant or *balanced*, equal to 1 on exactly half of the 2^n strings, and the task is to decide which. One quantum query decides it exactly (Example 5.4). An exact

classical algorithm needs $2^{n-1}+1$ queries on the constant 0: a run that stops after at most 2^{n-1} answers 0 gets the same answers, with the same random bits, from a balanced function that is 0 on every string asked, and gives it the same verdict. Against exact algorithms the gap is exponential. With bounded error it is not: ask three independent uniform strings, and say "constant" if the three values agree. A constant f always passes, and a balanced f gives three equal values with probability $2\cdot(\frac{1}{2})^3=\frac{1}{4}$; so three queries succeed with probability at least $\frac{3}{4}$ for every f, whatever n. From here on, fewer queries means fewer than a classical algorithm with bounded error needs.

5.2 The garbage guard

Example 1.11 left an ancilla holding x_1x_2; here is its cost. Let an oracle for Deutsch's f also write $g(x)$ on wire 3, $V|x\rangle|y\rangle|z\rangle=|x\rangle|y\oplus f(x)\rangle|z\oplus g(x)\rangle$ (a permutation), and run Deutsch's circuit with V:



Example 5.2 (QQ: garbage in Deutsch's circuit)**.** The query leaves

$$2^{-1/2}\textstyle\sum_x(-1)^{f(x)}|x\rangle|-\rangle|g(x)\rangle\,.$$

If g is constant, this is Example 2.6's state times a fixed $|c\rangle$, and b is read with certainty. If $g(x)=x$, Corollary 3.10 puts wires 1–2 in $\frac{1}{2}\mathbb{1}\otimes|-\rangle\langle-|$ for all four f; by (3.4), after the last H the top wire reads 0 or 1 with probability $\frac{1}{2}$ each.

In general, the corollary puts the query register of $\sum_x\alpha_x|x\rangle\otimes|g(x)\rangle$ in

$$\textstyle\sum_{g(x)=g(x')}\alpha_x\overline{\alpha_{x'}}\,|x\rangle\langle x'|\,. \tag{5.1}$$

Its off-diagonal entries, the cross terms of (2.5) in a reading after gates on the query register, survive only where $g(x)=g(x')$: garbage left alone destroys the interference between the branches it tells apart, and only those. Every oracle from here on is clean (Theorem 1.9).

5.3 The Hadamard transform

On n qubits, Deutsch's Hadamards become $H^{\otimes n}$, whose entries are signs. For $x,y\in\{0,1\}^n$ write $x\cdot y=\sum_i x_iy_i \bmod 2$.

Proposition 5.3 (The Hadamard transform)**.** *For every $x\in\{0,1\}^n$,*

$$H^{\otimes n}|x\rangle=2^{-n/2}\textstyle\sum_y(-1)^{x\cdot y}|y\rangle\,, \tag{5.2}$$

and for every $y\neq 0$,

$$\textstyle\sum_x(-1)^{x\cdot y}=0. \tag{5.3}$$

Proof. The identities $H|0\rangle=|+\rangle$ and $H|1\rangle=|-\rangle$ give $H|x_i\rangle=2^{-1/2}\sum_{y_i}(-1)^{x_iy_i}|y_i\rangle$. By the interchange law, $H^{\otimes n}|x\rangle=H|x_1\rangle\otimes\cdots\otimes H|x_n\rangle$; expanding this product of sums gives one

term for each $y = y_1 \cdots y_n$, with the coefficient $2^{-n/2} \prod_i (-1)^{x_i y_i} = 2^{-n/2}(-1)^{x \cdot y}$. For $y \neq 0$, take $y_i = 1$ and let e_i be the string with a 1 in place i only: $x \mapsto x \oplus e_i$ pairs the strings and changes $x \cdot y$ by 1, so the terms cancel in pairs. ∎

The signs are characters of the group $\mathbb{Z}_2^n$ of strings under $\oplus$: $\chi_y(x) = (-1)^{x \cdot y}$ has $\chi_y(x \oplus x') = \chi_y(x)\chi_y(x')$, and it is the product over the coordinates of the characters $(-1)^{x_i y_i} = e^{2\pi i x_i y_i / 2}$ of the cyclic group $\mathbb{Z}_2$ (*Symmetries*, L3). Since $\chi_y \chi_{y'} = \chi_{y \oplus y'}$, (5.3) is their orthogonality (*Symmetries*, L5). So $H^{\otimes n}$ is the Fourier transform of $\mathbb{Z}_2^n$, and a reading after it is *Fourier sampling*; Lecture 7 does it on $\mathbb{Z}_N$.

Example 5.4 (Q: Deutsch–Jozsa)**.** In Example 2.6's circuit with n query qubits, the kickback and (5.2) give $0 \cdots 0$ the amplitude $2^{-n} \sum_x (-1)^{f(x)}$: ± 1 if f is constant, 0 if it is balanced. One query decides.

5.4 Simon and the rank test

Definition 5.5 (Simon's problem)**.** A function $f\colon \{0,1\}^n \to \{0,1\}^n$ has *Simon's promise* with the string $s \in \{0,1\}^n$ if, for all x and x',

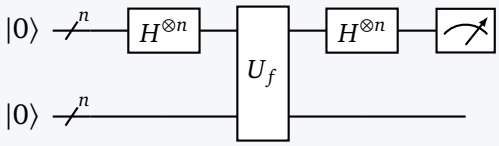
$$f(x) = f(x') \iff x' \in \{x, x \oplus s\}.$$

Simon's problem is to decide, given the oracle of such an f, whether $s = 0$.

If $s = 0$ the promise says that f is injective (fewer output bits would rule this out); if $s \neq 0$, that each value of f is taken on exactly one pair $\{x, x \oplus s\}$. Either way f determines s; finding s is the search version. Write $s^\perp = \{y : y \cdot s = 0\}$. It is all of $\{0,1\}^n$ if $s = 0$, and by (5.3), with the roles of x and y exchanged, exactly half of it, 2^{n-1} strings, if $s \neq 0$.

Strings are vectors over the field $\mathbb{F}_2 = \{0,1\}$, with $\oplus$ as their sum and $x \cdot y$ as a dot product. Gaussian elimination, the rank, and the rank–nullity theorem hold over $\mathbb{F}_2$ as over $\mathbb{R}$, since they use only the field operations.

Theorem 5.6 (Simon's algorithm)**.** *Let f have Simon's promise with s. A run of the circuit*



where $|0\rangle$ is the string of zeros, makes one query, and:

- *(i) it returns y with probability $1/|s^\perp|$ if $y \in s^\perp$, and 0 otherwise;*
- *(ii)* (the rank test) *let l runs return $y_1, \ldots, y_l$, and let r be the rank over $\mathbb{F}_2$ of the matrix with the rows y_j. If $s \neq 0$, then $r \leq n-1$. If $s = 0$, then $r = n$ with probability at least $1 - (2^n - 1)/2^l$, which is above $\frac{3}{4}$ for $l = n+2$;*
- *(iii) if $s \neq 0$, the y_j span $s^\perp$ with probability at least $1 - (2^{n-1} - 1)/2^l$, and when they do, s is the only $z \neq 0$ with $y_j \cdot z = 0$ for every j.*

So the verdict "$s = 0$ if and only if $r = n$" after $n+2$ runs decides Simon's problem with $n+2$ queries; it errs only if $s = 0$, with probability below $\frac{1}{4}$.

Proof of (i). By (5.2) with $x = 0$, the first $H^{\otimes n}$ makes $2^{-n/2} \sum_x |x\rangle |0\rangle$, and the query makes

$2^{-n/2}\sum_x |x\rangle\,|f(x)\rangle$. By (5.2) again, the second $H^{\otimes n}$ makes

$$\begin{aligned} &2^{-n}\textstyle\sum_{x,y}(-1)^{x\cdot y}\,|y\rangle\,|f(x)\rangle \\ &\quad = \textstyle\sum_y |y\rangle \otimes \varphi_y, \end{aligned}$$

with $\varphi_y = 2^{-n}\sum_x(-1)^{x\cdot y}\,|f(x)\rangle$. By (3.2), the reading gives y with probability $\|\varphi_y\|^2$. Collect the terms of φ_y by the value $z = f(x)$, and let c_z be the sum of $(-1)^{x\cdot y}$ over the x with $f(x) = z$; the $|z\rangle$ are orthonormal, so $\|\varphi_y\|^2 = 2^{-2n}\sum_z |c_z|^2$. If $s = 0$, each of the 2^n values has one such x, so $|c_z| = 1$ and $\|\varphi_y\|^2 = 2^{-n}$. If $s \neq 0$, each of the 2^{n-1} values has two, x and $x \oplus s$, and since $(x \oplus s)\cdot y = x\cdot y + s\cdot y$,

$$c_z = (-1)^{x\cdot y}\bigl(1 + (-1)^{s\cdot y}\bigr),$$

which is 0 if $s\cdot y = 1$ and ± 2 if $s\cdot y = 0$. So $\|\varphi_y\|^2 = 2^{-2n}\cdot 2^{n-1}\cdot 4 = 2^{-(n-1)}$ on $s^\perp$, and 0 off it. ∎

The second register is never read, and it matters. It holds $f(x)$, which tells apart the branches with different values: by (5.1) with $g = f$, only x and $x \oplus s$ still interfere, and their cancellation when $s\cdot y = 1$ is the whole effect. An oracle that also left x behind would tell every branch apart, and every y would have probability 2^{-n}, whatever s is.

Lemma 5.7 (Random vectors span). *Let $V \subseteq \mathbb{F}_2^n$ be a subspace of dimension k, and let $y_1, \dots, y_l$ be independent and uniform in V. They span V with probability at least $1 - (2^k - 1)/2^l$.*

Proof. A basis $v_1, \dots, v_k$ of V gives a linear bijection $a \mapsto \sum_i a_i v_i$ of $\mathbb{F}_2^k$ onto V. Write y_j as the image of a_j: the a_j are independent and uniform in $\mathbb{F}_2^k$, and they span it exactly when the y_j span V. Let A be the $l \times k$ matrix with the rows a_j. By rank–nullity, the a_j fail to span $\mathbb{F}_2^k$ exactly when $Ac = 0$ for some $c \neq 0$, that is, when $a_j\cdot c = 0$ for every j. For a fixed $c \neq 0$, (5.3) says that $a\cdot c = 0$ for exactly half of the $a \in \mathbb{F}_2^k$, so all l of the a_j have $a_j\cdot c = 0$ with probability 2^{-l}. There are $2^k - 1$ such c, and the probability that one of these events occurs is at most the sum of their probabilities. ∎

Proof of (ii) and (iii). Separate runs act on fresh registers, so by (3.2) their outcomes are independent, each uniform in $s^\perp$ by (i). The set $s^\perp$ is a subspace, the null space of the $1 \times n$ matrix s: of dimension n if $s = 0$, and $n - 1$ if $s \neq 0$. (ii) If $s \neq 0$, every y_j lies in $s^\perp$, so $r \leq n - 1$. If $s = 0$, the lemma with $V = \mathbb{F}_2^n$ gives $r = n$ with the probability stated; for $l = n + 2$ it is above $1 - 2^n/2^{n+2} = \frac{3}{4}$. (iii) The lemma with $V = s^\perp$, $k = n - 1$, gives the probability. If the y_j span $s^\perp$, the matrix with the rows y_j has rank $n - 1$, so its null space, the z with $y_j\cdot z = 0$ for every j, has dimension 1 and two elements. It contains 0 and s. ∎

The rank and the null space come from Gaussian elimination, $O(n^3)$ operations on bits. With $l = n + 1$, (iii) finds s with probability above $\frac{3}{4}$.

5.5 An example: $n = 3$, $s = 110$

Example 5.8 (Q: Simon's circuit for $n = 3$). The strings pair up as $\{000, 110\}$, $\{001, 111\}$, $\{010, 100\}$ and $\{011, 101\}$, and f takes the values 000, 001, 010 and 100 on these pairs, in this order; it has Simon's promise with $s = 110$. After the first Hadamards and the query, the state is

$$2^{-3/2}\textstyle\sum_{\text{pairs}}\bigl(|x\rangle + |x \oplus 110\rangle\bigr)\,|f(x)\rangle\,,$$

one term for each pair, with x either of its strings. Here $y \cdot s = y_1 \oplus y_2$, and (5.2) gives, for each pair,

$$H^{\otimes 3} \frac{|x\rangle + |x \oplus 110\rangle}{\sqrt{2}} = \frac{1}{2} \sum_{y \in s^\perp} (-1)^{x \cdot y} |y\rangle ,$$

with $s^\perp = \{000, 001, 110, 111\}$: the terms with $y_1 \neq y_2$ cancel within the pair. So for $y \in s^\perp$, $\varphi_y = \frac{1}{4} \sum_{\text{pairs}} (-1)^{x \cdot y} |f(x)\rangle$, four orthogonal terms of length $\frac{1}{4}$: the reading returns 000, 001, 110 or 111, each with probability $\frac{1}{4}$.

The state after the query holds all eight values $f(x)$, and it is tempting to read it as eight evaluations at once. A run returns one string y, uniform among four, and no value of f; s shows only over several runs, in which strings never occur.

The rank test: two runs that return 001 and 110 have rank 2 and span $s^\perp$, and $z \cdot 001 = z \cdot 110 = 0$ leaves $z = 000$ or $z = 110$; so $s = 110$. Any five runs have rank at most 2, and the verdict is $s \neq 0$, correctly. Had f been injective, five runs would have had rank 3 with probability at least $1 - 7/2^5 = \frac{25}{32}$, by (ii).

5.6 The classical bound

A classical algorithm learns that $s \neq 0$ from a *collision*: two queried strings $x \neq x'$ with $f(x) = f(x')$, and then $s = x \oplus x'$.

> **Theorem 5.9** (Simon's classical bound · *cited*)**.** *There is a constant $c > 0$ such that every classical query algorithm that decides Simon's problem with bounded error makes at least $c\, 2^{n/2}$ queries.*

This is the asymptotic lower bound denoted by $\Omega(2^{n/2})$.

The proof is de Wolf's (§3.3.2, pp. 25–26), for this decision version. The idea, for s and f chosen at random, is that until two queries collide, the answers are distinct strings that look the same whether $s = 0$ or not. And $T < 2^{n/2}$ queries collide with probability at most $T^2/2^n$, since a new x collides only if s is one of the fewer than T strings $x \oplus x'$ with x' asked before, while s is still spread over about 2^n candidates. Against the $n + 2$ queries of Theorem 5.6, the gap is exponential.

5.7 Status

Theorems 5.6 and 5.9 compare numbers of queries, and the comparison has a status:

- it is *proved*: both bounds are theorems, the classical one for every algorithm, however much it computes between queries;
- it is *relative to an oracle*: both sides see f only through its oracle;
- it says *nothing about* BPP *versus* BQP, the decision problems that classical and quantum computers solve efficiently with bounded error (Lecture 6). A problem given by a circuit for f, rather than by its oracle, hands a classical algorithm the circuit itself, and the lower bound no longer applies.

De Wolf (pp. 17 and 25) stresses that the separation is of queries, not of operations in the circuit model. Relative to suitable oracles BPP and BQP are known to differ, while without

an oracle no separation is known (Watrous 2008, p. 18); Lecture 6 says why a proof would be hard.

Summary.

- *The query model* compares query counts: Deutsch–Jozsa takes one quantum query, $2^{n-1}+1$ exact classical ones, but at most three with bounded error (Definition 5.1 and Example 5.4).
- *The guard:* garbage left alone destroys the interference between the branches it tells apart, and only those (Example 5.2).
- *The main result:* Simon's Fourier sampling returns uniform y with $y \cdot s = 0$. The rank test after $n+2$ runs decides whether $s = 0$, with error probability below $\frac{1}{4}$; it errs only when $s = 0$ (Theorem 5.6). Classically $\Omega(2^{n/2})$ queries are needed (Theorem 5.9, cited): a separation proved, relative to an oracle.
- *Next:* Lecture 6, the circuit model.

References

De Wolf, p. 14, §2.4, pp. 17–19, and §§3.1–3.3, pp. 23–27. Watrous (2025), §5.1, pp. 127–133, §5.4, pp. 147–154, and §6.3, pp. 180–184. Nielsen and Chuang, §§1.4.2–1.4.4, pp. 30–36. Kitaev, Shen and Vyalyi, §13.1, pp. 116–119. Watrous, *Quantum computational complexity* (2008), p. 18. D. Deutsch and R. Jozsa, Proc. R. Soc. Lond. A **439** (1992), 553–558. D. R. Simon, SIAM J. Comput. **26** (1997), 1474–1483.